

DrivebuddyAI / Roadzen Technologies — Data Scientist Campus Placement Challenge

Data Analysis Report: Pav Bhaji vs. Not Pav Bhaji — Text-Based Instagram Post Classification

Author: Hardik

Primary source: `src/DrivebuddyAI_PavBhaji_Challenge.py`

Notebook: `notebooks/DrivebuddyAI_PavBhaji_Challenge.ipynb`

1. Executive Summary

This project solves the DrivebuddyAI challenge as a **text-based binary classification problem**. The image folders are used only to establish the ground-truth label; no image pixels, CNNs, OCR, image embeddings, or computer-vision features are used for modeling.

The final labeled dataset contains **452 posts: 269 Not Pav Bhaji (59.5%) and 183 Pav Bhaji (40.5%)**. All 452 labeled image files were deterministically matched to JSON metadata records.

A key dataset issue was identified: the direct Pav-Bhaji target terms occur in almost all examples of both classes. To avoid letting the classifier memorize the target vocabulary, the final modeling pipeline performs **strict target-term removal** rather than replacing the target with a placeholder. In addition, the train/test split and model-selection cross-validation are **group-aware**, preventing both owner-level leakage and exact normalized-caption duplication across folds.

Three classical models were evaluated. On grouped 5-fold CV, Logistic Regression achieved the highest mean F1 (**0.591**) and was therefore selected as the final model. On the untouched held-out test set, the final model achieved **0.598 accuracy, 0.500 precision, 0.541 recall, 0.519 F1, and 0.630 ROC-AUC**.

These metrics should be treated as an honest baseline for a relatively small dataset rather than as a production-ready performance estimate.

2. Problem Definition

Task: Predict whether an Instagram post is Pav Bhaji (label 1) or Not Pav Bhaji (label 0).

Challenge constraint: The task is explicitly text-based. Images are not model inputs.

Required submission: Source code and data analysis report.

3. Dataset Description

The supplied archive contains:

```
dataset/  
|-- images/  
|   |-- 0/    -> Not Pav Bhaji  
|   `-- 1/    -> Pav Bhaji  
`-- pavbhaji.json
```

The JSON contains Instagram post metadata. The joined modeling fields include caption text, hashtags, likes, comment counts, location presence, video status, timestamp-derived information, and simple text statistics.

The final labeled dataset is **452 rows** because only 452 JSON records correspond to the supplied labeled image files. The remaining scraped records are not given a ground-truth label and are therefore excluded from supervised evaluation rather than guessed.

4. JSON-to-Image Label Mapping

The mapping is deterministic:

1. Extract the image filename embedded in the JSON image URL fields.
2. Match that filename to the provided `images/0` and `images/1` folders.
3. Assign the folder name as the binary label.

Validation:

Check	Result
Labeled image files	452
Successfully matched to JSON	452 / 452 (100%)
Unmatched labeled images	0
Ambiguous filename matches	0
Final usable labeled samples	452

No manual labels were introduced.

5. Data Quality and Leakage Investigation

Class distribution

Class	Count	Percentage
0 — Not Pav Bhaji	269	59.5%
1 — Pav Bhaji	183	40.5%

Target-token investigation

The direct target vocabulary is present in nearly all posts:

Class	Posts containing target-token variant
Not Pav Bhaji	99.6%
Pav Bhaji	100.0%

Because the target phrase appears in both classes, its simple presence/absence is not sufficient by itself. However, target-specific n-grams and repeated target-containing phrases can still become shortcuts. The project therefore includes a naive baseline for comparison but uses a stricter representation for the submitted final model.

Other leakage checks

- **Duplicate JSON record IDs:** 0 within the 452-row labeled dataset.
- **Duplicate caption groups:** 4 groups involving 15 rows.
- **Owners with multiple posts:** 30 owners; maximum 8 posts from one owner.
- **Owner overlap between train and test:** 0.
- **Exact normalized-caption overlap between train and test:** 0.
- **Composite group overlap:** 0.

The composite grouping connects owner identity and duplicate normalized-caption groups, so either source of repeated information stays within a single split/fold.

6. Data Preprocessing

Text preprocessing is deterministic and reusable:

1. lowercase text;
2. remove URLs and `@mentions`;
3. replace emoji characters with a neutral `<emoji>` token;
4. remove the `#` marker while retaining hashtag words;

5. normalize punctuation/whitespace;
6. create a naive text version for the leakage probe;
7. create a **strict text version** in which direct target terms and their common spacing/hyphen/underscore variants are removed.

Numeric fields are coerced to numeric types. Legitimate absent `video_view_count` values for non-video posts are represented as zero. `likes` is median-imputed only if residual nulls occur.

All learned transformations such as TF-IDF vocabulary construction are fitted only inside the sklearn pipeline on the training folds, preventing vectorizer leakage.

7. Features Used

Text features

TF-IDF word unigrams and bigrams (`ngram_range=(1, 2)`) with sublinear term frequency.

Structured features

- `likes`
- `comments_count`
- number of tags
- number of hashtags in caption
- number of mentions
- number of URLs
- number of emojis
- caption character length
- caption word length
- `is_video`
- `has_location`

Explicitly excluded

`owner_id`, `record_id`, `shortcode`, filenames, raw image URLs, and image pixels are not model features. `owner_id` is used only for grouping. Comment `text` is not used because the supplied JSON contains comment counts rather than comment bodies.

8. Exploratory Data Analysis

The project generates seven EDA figures covering:

- class distribution,
- missing values,
- caption-length distribution,
- engagement distributions,
- common words,
- top hashtags,
- class-wise structured-feature comparisons.

The EDA shows that no single structured variable cleanly separates the two classes. Textual style and food-related vocabulary provide more useful signals, which is consistent with the challenge being a text classification task.

9. Modeling Methodology

The main modeling pipeline is:

```
caption + hashtags
  ↓
text cleaning / strict target removal
  ↓
TF-IDF (1-2 grams)
```

↓
combined with standardized numeric metadata
↓
classifier

Candidate classifiers:

- Logistic Regression (`class_weight='balanced'`)
- Linear SVM (`class_weight='balanced'`)
- Multinomial Naive Bayes (text-only pipeline because Naive Bayes requires non-negative features)

Split and validation strategy

The final train/test split uses **StratifiedGroupKFold** with the composite owner/caption grouping strategy. Model selection uses **5-fold StratifiedGroupKFold on the training set only**.

The held-out test set is not used for model tuning.

10. Cross-Validation Results

Model	CV Accuracy	CV Precision	CV Recall	CV F1	CV ROC-AUC
Logistic Regression	0.622 ± 0.037	0.528 ± 0.037	0.678 ± 0.082	0.591 ± 0.044	0.688 ± 0.040
Linear SVM	0.633 ± 0.036	0.541 ± 0.041	0.650 ± 0.057	0.590 ± 0.039	0.695 ± 0.054
Multinomial NB	0.647 ± 0.044	0.612 ± 0.088	0.342 ± 0.111	0.431 ± 0.110	0.670 ± 0.071

Selection: Logistic Regression because it has the highest mean grouped-CV F1 in the corrected pipeline.

11. Final Held-Out Test Evaluation

The final test set contains **92 posts** and has no owner overlap or exact normalized-caption overlap with training.

Metric	Final Logistic Regression
Accuracy	0.598
Precision	0.500
Recall	0.541
F1	0.519
ROC-AUC	0.630
PR-AUC	0.497

Confusion matrix

	Predicted Not PB	Predicted Pav Bhaji
Actual Not PB	35	20
Actual Pav Bhaji	17	20

12. Naive vs. Leakage-Controlled Baseline

A naive, unmasked Logistic Regression baseline is retained only as a diagnostic experiment.

Model	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC
Naive unmasked LogReg	0.620	0.524	0.595	0.557	0.648	0.513
Final strict Logistic Regres- sion	0.598	0.500	0.541	0.519	0.630	0.497

The controlled model is intentionally more conservative: its text representation excludes direct target vocabulary instead of learning from the target string itself.

13. Error Analysis

The corrected run produced **20 false positives** and **17 false negatives** on the held-out test set.

Common failure modes include:

- generic foodie language that resembles the positive class;
- other Indian street-food vocabulary that is textually adjacent to Pav Bhaji;
- long captions where the discriminative context is diluted across many unrelated tokens;
- dataset-specific influencer/hashtag writing styles.

The complete examples are stored in `outputs/error_analysis.csv`.

14. Model Interpretability

The final Logistic Regression is interpretable through its text and numeric coefficients. The strongest positive textual signals in the corrected run include terms such as `foodlover`, `foodgasm`, `foodphotography`, `foodstagram`, `buttery`, and `indianstreetfood`. Strong negative signals include terms such as `chef`, `vadapav`, `panipuri`, `gobimanchurian`, and `bhelpuri`.

These coefficients are **correlations learned from this dataset**, not causal effects and not universal rules about Pav Bhaji.

Critically, the final coefficient list contains **no direct pav, bhaji, pavbhaji, or <target_masked> features** after strict target removal.

15. Reproducibility

The standalone script is the canonical executable pipeline. Run:

```
cd src
python DrivebuddyAI_PavBhaji_Challenge.py --data-dir ../dataset --output-dir ../outputs
```

The notebook wraps the same pipeline for presentation and review.

All reported metrics in this document were regenerated from the corrected run and saved in `outputs/`.

16. Limitations

1. Only 452 labeled posts are available, so model estimates have noticeable sampling variance.
2. The JSON does not provide comment bodies, so comment-text modeling is impossible from the supplied data.
3. Some useful predictive terms may reflect the specific Instagram community represented in this scrape and may not generalize.
4. The strict leakage policy removes potentially informative direct target vocabulary, which is deliberate for this challenge's evaluation integrity.

17. Possible Improvements

- collect more independently labeled data;
- test character-level n-grams for spelling/spacing variation;
- tune SVM/Logistic Regression hyperparameters inside the grouped CV framework;
- investigate semi-supervised use of the unlabeled JSON records only after establishing a reliable labeling strategy;
- evaluate the final system on a separately collected external dataset.

18. Submission Checklist

Primary submission: notebooks/DrivebuddyAI_PavBhaji_Challenge.ipynb

Source code: src/DrivebuddyAI_PavBhaji_Challenge.py

Data analysis report: reports/DrivebuddyAI_Data_Analysis_Report.pdf and .md

No Streamlit application is required by the challenge deliverables; this package intentionally focuses on the requested ML source code and analysis report.